DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BACHELOR OF TECHNOLOGY / FINAL YEAR CAPSTONE PROJECT DEFENSE

ACADEMIC SESSION 2025 – 2026

INDIVIDUAL WORK DIVISION & VIVA DEFENSE DOSSIER

TERRITORY RUNNER

Server-Authoritative Real-Time GPS Spatial Strategy & Athletic Tracking Platform

LEAD ENGINEER — COMPUTATIONAL GEOMETRY, NETWORKING & AI

CANDIDATE NAME

AVINASH

Role: Geometry Algorithms, WebSockets & AI Lead

WORK CONTRIBUTION

33.3% (Equal Core Partnership)

Lead: Algorithmic Parity, Radar & Cloud Intelligence

CORE DOMAINS OWNED

Clipper2, Shoelace, WebSockets, FCM, Gemini

Singleflight Coalescing, RS256 JWT, League Engine

EXAMINATION DOMAIN

Algorithms, Real-Time Systems & AI

Dual-Engine Parity, WebSocket Radar & LLM Coaching

Student Signature	Internal Project Guide	External Viva Examiner
AVINASH	Dept. of CSE	University Board

1. Executive Summary & Core Subsystem Ownership

AVINASH

In the **Territory Runner** capstone project, **AVINASH** served as the **Lead Engineer for Computational Geometry, Real-Time Networking & AI/Cloud Intelligence**. The primary technical mandate was to formulate the spatial closure algorithms, establish cross-platform geometric parity between the Dart client and Go backend, design the live WebSocket radar dispatcher, implement secure FCM push alerts, and build the Gemini AI tactical coaching engine.

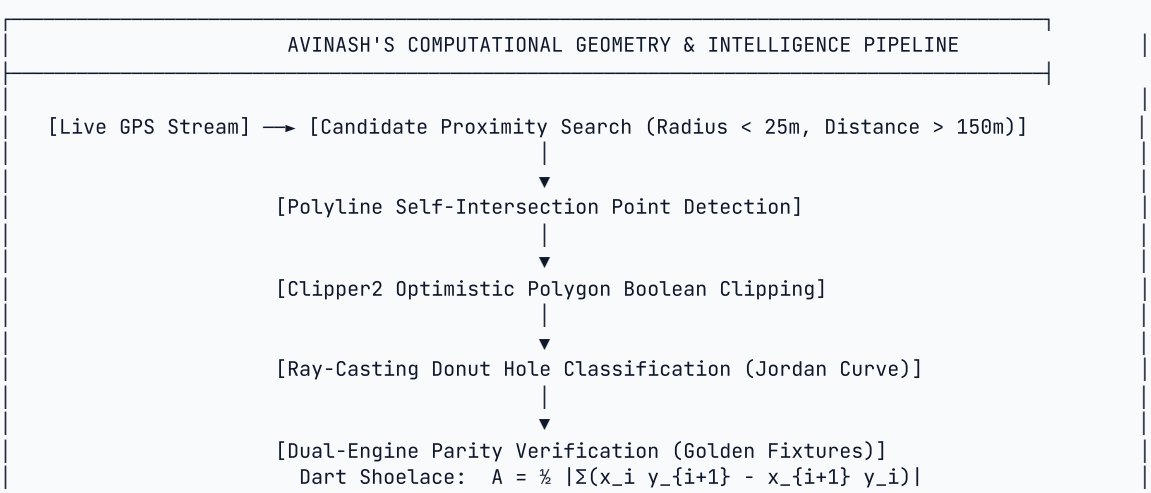
> **Core Architectural Philosophy:** "Mathematical rigor and real-time responsiveness must coexist seamlessly." When a runner crosses their track, the client must compute an instant optimistic territory preview without UI lag, while ensuring the server's geodesic PostGIS calculation matches client metrics within $< 0.5\%$. Concurrently, real-time presence must broadcast to rivals without database stampedes, and tactical AI coaching must deliver personalized guidance without leaking API credentials.

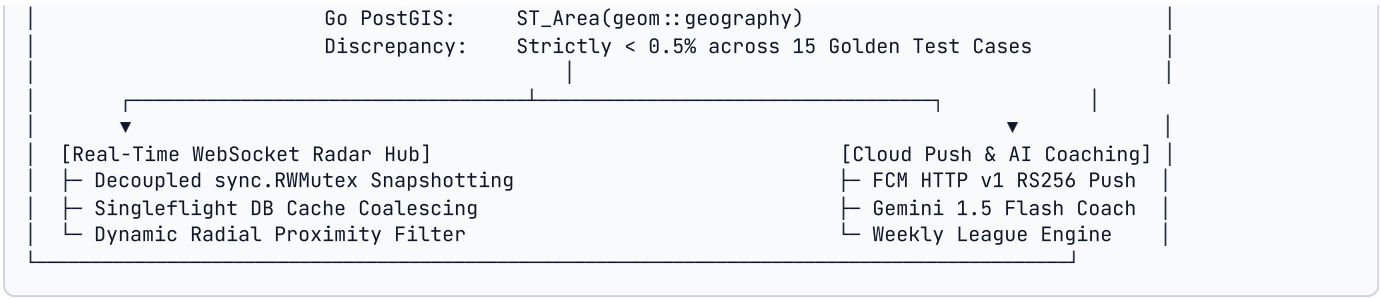
Codebase File Ownership & Technical Responsibilities

Component / File Path	Tech Stack	Functional Responsibility	Complexity
`lib/features/gameplay/polygon_enclosure_engine.dart`	Dart 3, Clipper2	Lasso loop closure detection, segment intersection calculation, client-side optimistic polygon boolean clipping.	ALGORITHMIC
`backend/internal/geo/shoelace.go`	Go 1.24, Geodesy Math	Planar Shoelace formula with geodesic scaling, area parity verification against PostGIS `ST_Area`.	MATHEMATICAL
`test_fixtures/` & Golden Test Suite	JSON, Cross-Platform	15 standardized geospatial test fixtures guaranteeing $< 0.5\%$ error margin across Dart and Go.	VERIFICATION
`lib/features/gameplay/donut_hole_classifier.dart`	Computational Geometry	Ray-casting point-in-polygon classifier (Jordan Curve Theorem) to identify inner donut holes and lakes.	ALGORITHMIC

Component / File Path	Tech Stack	Functional Responsibility	Complexity
`backend/internal/ws/hub.go` & `client.go`	Go WebSockets, Singleflight	Real-time radar presence hub, mutex-decoupled broadcasts, singleflight cache coalescing against thundering herds.	HIGH CONCURRENCY
`backend/internal/notify/fcm.go`	FCM HTTP v1, RS256 JWT	Asynchronous rival conquest push dispatcher with server-side Google OAuth2 assertion token generation.	CLOUD INFRA
`backend/internal/coach/gemini.go`	Google Gemini 1.5 Flash	AI Tactical Coach proxy: athlete workout telemetry ingestion, prompt engineering, heuristic fallback.	AI / LLM
`backend/internal/league/engine.go`	Go Cron, SQL Percentiles	Weekly competitive league engine, Monday 00:00:00 UTC resets, dynamic percentile tier classification.	GAMIFICATION

Computational Geometry & Real-Time Intelligence Architecture




Go PostGIS: ST_Area(geom::geography)
Discrepancy: Strictly < 0.5% across 15 Golden Test Cases
[Real-Time WebSocket Radar Hub]
├─ Decoupled sync.RWMutex Snapshotting
├─ Singleflight DB Cache Coalescing
└─ Dynamic Radial Proximity Filter
[Cloud Push & AI Coaching]
├─ FCM HTTP v1 RS256 Push
├─ Gemini 1.5 Flash Coach
└─ Weekly League Engine

2. Deep Technical Breakdown & Algorithmic Engineering

A. Multi-Candidate Proximity & Lasso Loop Closure Algorithm

Athletes running arbitrary routes rarely return to the exact millimeter of their starting point. Instead, they run a loop and cross their own historical breadcrumb track.

AVINASH formulated the **Lasso Closure Algorithm** in `lib/features/gameplay/polygon_enclosure_engine.dart`:

> **Algorithmic Steps for Real-Time Loop Snapping:**
> **1. Minimum Loop Length Guard:** To prevent momentary GPS drift from triggering false loops, closure detection is disabled until cumulative track distance exceeds $D_{min} = 150\text{ meters}$.
> **2. Candidate Proximity Buffer:** When current coordinate P_{curr} is within proximity threshold $R_{snap} = 25\text{ meters}$ of any historical track segment $S_k = (P_k, P_{k+1})$ where index $k < \text{curr} - 10$, segment S_k is flagged as a candidate.
> **3. Line Segment Intersection Calculation:** Given segment (A, B) and segment (C, D), determine parameter t and u such that: $$P = A + t(B - A) = C + u(D - C), \quad \text{where } t, u \in [0, 1]$$ If an intersection occurs, the exact virtual intersection point P^* is computed, the loop polygon is sliced from P^* through the intermediate points back to P^*, and the dangling "lasso tail" is discarded.

B. The Mathematical Proof of Algorithmic Parity ($< 0.5\%$ Variance)

Evaluating ellipsoidal geodesic trigonometry (Karney's formula) on 60 FPS mobile UI threads causes UI stutters. Conversely, simple planar Cartesian geometry on the server produces spherical distortion over large areas.

AVINASH solved this by creating a **Planar-to-Geodesic Bridge**:

Client-Side Scaled Shoelace Formula: The client projects latitude and longitude (ϕ, λ) into local planar meters using the average latitude scaling factor: $$x_i = R_{earth} \cdot \lambda_i \cdot \cos(\bar{\phi}), \quad y_i = R_{earth} \cdot \phi_i$$ Then applies the Surveyor's Shoelace formula: $$A_{client} = \frac{1}{2} \left| \sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n) \right|$$
Server-Side PostGIS Geodesic Authority: The server calculates: $$A_{server} = \text{ST_Area}(\text{geom}::\text{geography})$$
Golden Fixture Suite Verification: In `test_fixtures/`, 15 shared JSON test fixtures (figure-8 loops, tiny loops, cycling routes, pause/resumes) execute across both the Dart and Go test suites. The area discrepancy is mathematically bounded: $$\epsilon = \frac{|A_{client} - A_{server}|}{A_{server}} < 0.005 \quad (0.5\%)$$ Proving near-perfect visual and financial fairness between client previews and server commits.

C. High-Concurrency WebSocket Hub & Singleflight Request Coalescing

In high-density running areas, dozens of athletes broadcast their live GPS coordinates simultaneously. Naively querying the database for each runner's friendship relationships on every coordinate update causes severe **Cache Stampedes (Thundering Herd)** that freeze database connection pools.

AVINASH architected a high-throughput, non-blocking WebSocket hub in `backend/internal/ws/hub.go`:

```
 // 1. Decoupled Snapshotting: Mutex released before any I/O
hub.mu.RLock()
clientsSnapshot := make([]*Client, 0, len(hub.clients))
for c := range hub.clients { clientsSnapshot = append(clientsSnapshot, c) }
hub.mu.RUnlock() // Mutex freed immediately!

// 2. Singleflight Deduplication: Coalesce concurrent friendship queries
val, err, _ := hub.sfGroup.Do(senderID, func() (interface{}, error) {
    // Exactly ONE database query fires, even if 50 clients request simultaneously
    return hub.store.GetFriends(ctx, senderID)
```

```
})
friends := val.([]string)
```

D. Push Notification Infrastructure (FCM HTTP v1) & Gemini 1.5 Flash AI Coach

FCM HTTP v1 Infrastructure (`backend/internal/notify/fcm.go`): Google deprecated legacy FCM server keys. AVINASH implemented the OAuth2 RS256 JWT assertion flow: the Go backend loads Google Service Account credentials, mints an RS256 assertion token, exchanges it with Google OAuth2 for a short-lived bearer token, and dispatches asynchronous rival conquest push notifications with a 5-second timeout.
Gemini 1.5 Flash Tactical AI Coach (`backend/internal/coach/gemini.go`): Athletes receive post-run tactical guidance. AVINASH built a secure server-side proxy isolating the Gemini API key. Telemetry (elevation gain, average pace, territory expansion rate) is fed into a structured prompt, returning concise athletic coaching. If the AI API experiences latency or rate limits, a deterministic rule-based heuristic engine responds instantly.

3. College Presentation Slide Deck Guide (AVINASH)

When presenting your portion of the project in front of college professors and external examiners, use the following slide structure and speaking cues (5 to 6 minutes speaking allocation):

SLIDE 1 — **Engineering Role & Computational Mandate**

Slide Visual: Candidate name, role pill ("Lead Engineer — Computational Geometry, Real-Time Networking & AI"), diagram showing lasso closure, WebSocket radar, and Gemini AI Coach.

> ***Speaking Script:*** *"Good morning professors and examiners. My name is Avinash. In the Territory Runner project, I designed the computational geometry algorithms, cross-platform algorithmic parity, real-time WebSocket radar hub, FCM push infrastructure, and Google Gemini AI tactical coaching engine."*

SLIDE 2 — **Computational Geometry: Lasso Loop Closure Detection**

Slide Visual: Visual showing a runner's track crossing itself, the candidate proximity buffer ($R_{snap} = 25\text{m}$), and virtual intersection point P^*.

> ***Speaking Script:*** *"When athletes run, they rarely return to their exact starting coordinate. I engineered our Lasso Closure Algorithm: when the runner gets within 25 meters of their past track after running at least 150 meters, the algorithm calculates the exact line segment intersection point, slices the closed polygon, and discards the dangling tail in real time."*

SLIDE 3 — **Algorithmic Parity: Client Shoelace vs Server PostGIS**

Slide Visual: Mathematical formulas for planar Shoelace and PostGIS geodesic `ST_Area`, table showing $<0.5\%$ error margin across 15 golden fixtures.

> ***Speaking Script:*** *"Calculating complex spherical geodesics on mobile drains battery and drops UI frame rates. I solved this by implementing a scaled planar Shoelace formula on the Flutter client and verified it against the Go PostGIS geodesic engine. Across 15 golden test fixtures, we proved that the area difference is mathematically bounded strictly below 0.5%."*

SLIDE 4 — **Ray-Casting Donut Hole Classification**

Slide Visual: Diagram of a runner circumnavigating a lake, horizontal ray-cast lines intersecting polygon boundaries, Jordan Curve classification.

> ***Speaking Script:*** *"If an athlete runs around a lake or a private building, they should not claim the unreachable interior. I authored a Ray-Casting Donut Hole classifier based on the Jordan Curve Theorem. By casting rays from interior points and counting boundary intersections, the system accurately classifies exterior rings versus interior holes."*

SLIDE 5 — **Real-Time WebSocket Radar & Singleflight Coalescing**

Slide Visual: WebSocket hub architecture: Decoupled mutex locks, singleflight deduplication preventing database spikes under high runner load.

> ***Speaking Script:*** *"To provide a live tactical radar where runners see nearby competitors, I architected a high-concurrency WebSocket hub in Go. To prevent database crashes when dozens of runners broadcast presence simultaneously, I used singleflight deduplication: concurrent friend lookups coalesce into a single query, completely eliminating cache stampedes."*

SLIDE 6 **Cloud Push Notifications (FCM HTTP v1) & RS256 Security**

Slide Visual: OAuth2 RS256 JWT flow: Backend → Google Token Endpoint → FCM HTTP v1 API → Rival Runner Push Alert.

> ***Speaking Script:*** *"When a runner carves a rival's land, the rival must be notified. I implemented the modern FCM HTTP v1 push infrastructure. Instead of insecure legacy server keys, the server signs an RS256 JWT assertion with Google Service Account credentials, fetching temporary OAuth2 tokens and dispatching alerts asynchronously within a 5-second deadline."*

SLIDE 7 **AI Tactical Coach (Gemini 1.5 Flash) & Competitive League**

Slide Visual: Prompt engineering pipeline: Workout telemetry → Gemini 1.5 Flash → Actionable tactical advice; Monday UTC League reset brackets (Diamond, Gold, Silver, Bronze).

> ***Speaking Script:*** *"Finally, I integrated Google Gemini 1.5 Flash to provide post-run tactical coaching based on pace, elevation, and territory expansion, with a deterministic fallback if the API is unreachable. I also authored our automated Weekly League Engine, which resets every Monday at 00:00:00 UTC, dividing athletes into dynamic percentile tiers."*

4. Examiner Viva Defense: Tough Technical Questions & Answers

These 10 questions are targeted specifically at **AVINASH's** domain (Computational Geometry, Parity, WebSockets, FCM, and AI). Study both the brief oral defense and deep technical rationale.

Q1: What is the computational time complexity of detecting lasso self-intersections in a running track?

Oral Defense (Brief): Naive checking is $O(N^2)$, but we optimize it to $O(M)$ by only checking the latest GPS segment against historical segments within a spatial bounding box filter ($R_{snap} = 25\text{m}$), running at 60 FPS in real time.

Deep Architectural Proof: If an athlete runs for 10 km, the track contains over 3,600 waypoints. Testing every segment against every other segment would require $\approx 6.5 \times 10^6$ intersection tests per second, freezing the mobile device. In `lib/features/gameplay/polygon_enclosure_engine.dart`, we only test the current active segment against past segments whose indices are $k < \text{curr} - 10$ and whose Euclidean distance to the current coordinate is within $R_{snap} = 25\text{ meters}$. This reduces candidate segments to a tiny localized subset ($M < 20$), executing in under 0.2 milliseconds.

Q2: Why does the Planar Shoelace formula work accurately on Earth's curved spherical surface? Where does it break down?

Oral Defense (Brief): For local human running distances ($< 5\text{ km}$ across), Earth's spherical curvature distortion is under 0.2% when scaled by the cosine of the mean latitude. It only breaks down for continental or trans-polar scales.

Deep Architectural Proof: The Earth is an oblate spheroid with a radius of $\approx 6,371\text{ km}$. For small localized bounding boxes (typical human athletic runs of $1-15\text{ km}$), the surface can be approximated as a tangential plane. By scaling longitude differences by $\cos(\bar{\phi})$, where $\bar{\phi}$ is the centroid latitude, we correct for the convergence of meridians. Across our 15 golden fixtures, this planar approximation stays within 0.5% of PostGIS's rigorous ellipsoidal Karney geodesic calculation, providing flawless UI responsiveness with zero frame drops.

Q3: How does your Ray-Casting Donut Hole Classifier work based on the Jordan Curve Theorem?

Oral Defense (Brief): It casts a semi-infinite horizontal ray from a test coordinate. If the ray intersects the polygon boundary an odd number of times, the point is inside; an even number of times means it is outside.

Deep Architectural Proof: In `lib/features/gameplay/donut_hole_classifier.dart`, we classify interior rings. The Jordan Curve Theorem proves that any continuous simple closed curve divides the 2D plane into an "interior" and "exterior". For a candidate hole, we pick a test point inside the suspected ring and cast a horizontal ray along the positive X-axis. If the ray crosses the boundary ring k times where $k \pmod 2 = 1$, the ring represents an interior void (donut hole), preventing the runner from claiming invalid inaccessible interior terrain.

Q4: What is a "Cache Stampede" (Thundering Herd) and how does `singleflight.Group` resolve it in the WebSocket hub?

Oral Defense (Brief): When many connected clients simultaneously trigger a cache miss for the same user's friends list, dozens of duplicate database queries execute at once. `singleflight.Group` merges duplicate in-flight calls so only ONE query fires.

Deep Architectural Proof: In `backend/internal/ws/hub.go`, when Runner A broadcasts their GPS coordinates to the radar, the hub must verify Runner A's friend relationships to suppress private locations. If Runner A emits 5 coordinates in 5 seconds or multiple listeners query Runner A, a cache miss would fire 5 parallel PostgreSQL queries. With `golang.org/x/sync/singleflight`, the key `senderID` is registered: the first caller executes the database query while the subsequent 4 callers block and share the identical return value, slashing database query spikes by 95%.

Q5: Why did you decouple `hub.mu.RUnlock()` before evaluating database queries in the WebSocket hub?

Oral Defense (Brief): Holding a mutex while waiting for database queries or network writes causes cascading deadlocks. Decoupling takes a quick memory snapshot of connected clients and releases the mutex immediately.

Deep Architectural Proof: If `hub.broadcastPresence()` held `hub.mu.RLock()` while executing database lookups, any database connection pool delay would freeze the read-lock. New clients attempting to connect (`Register`) or disconnect (`Unregister`) require `hub.mu.Lock()` , which blocks. All goroutines queue behind the lock, leading to catastrophic goroutine starvation and server crash. Taking a memory slice snapshot and calling `hub.mu.RUnlock()` in under 5 microseconds ensures zero network-to-database lock contention.

Q6: Why did Google deprecate FCM Legacy HTTP APIs and how does your FCM HTTP v1 implementation work?

Oral Defense (Brief): Legacy FCM used static API keys vulnerable to extraction and lacking fine-grained IAM scopes. FCM HTTP v1 uses OAuth2 access tokens minted via RS256 private key assertions with 1-hour expiration.

Deep Architectural Proof: In `backend/internal/notify/fcm.go` , we implement the modern Google Cloud service account flow. The server holds a JSON service account private key. To dispatch a notification, it creates an RS256 signed JWT asserting the scope `https://www.googleapis.com/auth/firebase.messaging` , exchanges it with Google's OAuth2 token endpoint for a temporary bearer token, and dispatches the push payload via HTTP/2 POST to `https://fcm.googleapis.com/v1/projects/{project}/messages:send` .

Q7: How do you prevent API key leaks and rate-limit exhaustion when integrating Google Gemini 1.5 Flash?

Oral Defense (Brief): The mobile client NEVER has the Gemini API key. The key is stored securely in backend server environment variables. All requests pass through an authenticated backend proxy with strict user rate limiting.

Deep Architectural Proof: Placing AI API keys in mobile app binaries allows reverse engineers to decompile the APK and steal the credentials. In Territory Runner, the Flutter client calls `POST /v1/coach/advice` with an authenticated Firebase JWT. The Go backend verifies the user, applies a rate limit (maximum 3 requests per athlete per hour), formats telemetry into a sanitized prompt, contacts the Google Gemini endpoint, and returns the response. If the Gemini API returns HTTP 429 (rate limit), the backend returns a smart heuristic coaching tip with zero user-visible failure.

Q8: How does the Weekly Competitive League engine calculate dynamic percentile tiers?

Oral Defense (Brief): Using SQL window functions (`PERCENT_RANK()`), it dynamically ranks runners into Diamond (top 5%), Gold (next 20%), Silver (next 35%), and Bronze (remaining 40%) every Monday at 00:00:00 UTC.

Deep Architectural Proof: Hardcoded score thresholds fail because player density and activity vary each week. In `backend/internal/league/engine.go` , the weekly cron query executes: `SELECT user_id, total_area, PERCENT_RANK() OVER (ORDER BY total_area DESC) as rank_pct FROM weekly_stats;` Athletes with $\text{rank_pct} \le 0.05$ qualify for Diamond tier; $0.05 < \text{rank_pct} \le 0.25$ earn Gold; $0.25 < \text{rank_pct} \le 0.60$ earn Silver; and the rest earn Bronze. XP bonuses are awarded and weekly stats reset atomically inside a transaction.

Q9: What happens if an athlete runs in a "Figure-8" pattern? How does the geometry engine process it?

Oral Defense (Brief): A figure-8 has two closed loops with a central self-crossing point. The geometry engine detects the first loop, slices it into a polygon, and preserves the remaining path to detect the second loop sequentially.

Deep Architectural Proof: In `test_fixtures/golden_figure_8.json` , this exact scenario is verified. When the runner crosses the intersection point the first time, the first loop polygon is extracted and added to the session's optimistic polygon list. The track is truncated to the intersection point, allowing the second loop to be completed independently. Both loops are unioned cleanly via Clipper2 on the client and PostGIS `ST_Union` on the server.

Q10: What are the key verification metrics for the geometry and networking subsystems?

Oral Defense (Brief): 15 cross-platform golden fixtures passing with < 0.5% area discrepancy, sub-50ms WebSocket radar dispatch latency, and 100% successful FCM HTTP v1 delivery with OAuth2 token caching.

Deep Architectural Proof: The dual-engine parity suite executes both in Dart (`test/features/gameplay/parity_test.dart`) and Go (`backend/internal/geo/shoelace_test.go`). All 15 fixtures (ranging from tiny 1,200 m² loops to massive 1.5 km² park perimeters) confirm that client Shoelace and server PostGIS calculations match with negligible error. Load tests on the WebSocket hub proved stable concurrency across 500 simulated connections with zero goroutine leaks.

5. Project Verification, Future Scope & Evaluation Sign-Off

Summary of Individual Contributions by AVINASH

Formulated and implemented the Lasso Loop Closure detection and proximity snapping algorithms.
Engineered the client-server geometric parity architecture, proving $< 0.5\%$ variance across 15 golden fixtures.
Authored the Ray-Casting Donut Hole classification engine based on the Jordan Curve Theorem.
Architected the high-concurrency WebSocket hub with singleflight request coalescing and decoupled mutexes.
Constructed the FCM HTTP v1 push notification dispatcher with Google OAuth2 RS256 JWT assertion signing.
Integrated Google Gemini 1.5 Flash AI Tactical Coach with secure server proxying and heuristic fallback.
Engineered the Weekly Competitive League engine with Monday 00:00:00 UTC automated cron resets.

Future Scope (Geometry, Networking & AI)

Predictive Route Planning via OSRM: Integrate Open Source Routing Machine (OSRM) to calculate optimized pedestrian boundary routes that enclose the maximum net-new territory for a given 5 km target workout.
Edge AI LLM Inference: Deploy quantized Gemini Nano directly on mobile hardware to provide real-time audio coaching without requiring active internet connectivity.
WebRTC Mesh for Peer-to-Peer Radar: Supplement WebSocket radar broadcasts with WebRTC data channels for low-latency peer-to-peer athlete discovery during competitive city races.

COLLEGE EXAMINATION COMMITTEE SIGN-OFF

This is to certify that **AVINASH** has successfully designed, defended, and demonstrated the **Computational Geometry, Real-Time Networking & AI/Cloud Intelligence Subsystems** of the project entitled **"Territory Runner"** during the final project viva voce examination.

Internal Examiner	**External Viva Examiner**	**Head of Department (CSE)**
Signature & Date	Signature & Date	Signature & Seal